

Simulación de un Vehículo Autónomo Submarino con Arquitectura Distribuida ROS 2 y Motor de Renderizado en Primera Persona Estilo *DOOM*

Sergio — al428707@uji.es

Universitat Jaume I — Grado en Ingeniería

Asignatura: Aéreos y Submarinos

Febrero 2026

Resumen

El presente trabajo describe el diseño, implementación y validación de **DOOM AUV Sim**, un simulador completo de Vehículo Autónomo Submarino (AUV) construido sobre el *middleware* ROS 2 Humble y el lenguaje Python. El sistema integra un motor de renderizado en primera persona basado en *raycasting* 2.5D —inspirado en el videojuego *DOOM* (1993)—, un motor de física subacuática con inercia hidrodinámica y consumo de batería, un canal de comunicación acústica submarina con latencia y pérdida de paquetes modeladas según la fórmula de absorción de Thorp, y un módulo de autonomía *onboard* con arquitectura de control inspirada en la subsunción de Brooks. La arquitectura distribuida emplea seis nodos ROS 2 interconectados mediante aproximadamente veinte tópicos *Pub/Sub*, sin recurrir a Servicios ni Acciones, lo cual se justifica por la naturaleza asíncrona y con latencia variable del dominio submarino. El sistema fue desarrollado de manera iterativa a lo largo de múltiples ciclos de diseño-implementación-prueba, resolviendo progresivamente problemas de estabilidad del control de profundidad, alineación del brazo robótico, comunicación acústica realista y captura de objetivos. Los resultados demuestran que el simulador permite la recolección autónoma de cinco objetivos distribuidos aleatoriamente, la evasión reactiva de obstáculos dinámicos y estáticos, y la exportación del mapa explorado en formatos PCD, PNG y OBJ.

1. Introducción

Los Vehículos Autónomos Submarinos (AUV, *Autonomous Underwater Vehicles*) representan una de las plataformas robóticas más desafiantes en ingeniería. A diferencia de los robots terrestres o aéreos, los AUV operan en un entorno donde la comunicación electromagnética convencional —incluido el GPS— resulta inviable más allá de unos pocos metros de profundidad. La alternativa estándar, la comunicación acústica submarina, introduce latencias del orden de cientos de milisegundos a segundos y tasas de pérdida de paquetes significativas, limitando severamente la capacidad de teleoperación en tiempo real [1].

En este contexto, el desarrollo de simuladores que reproduzcan fielmente estas restricciones operativas constituye una necesidad crítica tanto para la formación de operadores como para la validación de algoritmos de autonomía. Sin embargo, la mayoría de simuladores de AUV existentes —tales como UUV Simulator [2] o Stonefish [3]— se centran en la fidelidad física del medio subacuático o en la integración con marcos de planificación como Nav2, pero no necesariamente exponen al usuario las limitaciones perceptuales y comunicacionales del robot de manera intuitiva.

El presente proyecto propone un enfoque alternativo: un simulador de AUV que utiliza un motor de renderizado en primera persona inspirado en el clásico *DOOM* (id Software, 1993) para representar de forma inmersiva la perspectiva limitada del sonar frontal del vehículo. Este *raycasting* 2.5D, combinado

con un sistema de *fog of war* (niebla de guerra), obliga al operador a experimentar las mismas restricciones sensoriales que el AUV real, facilitando la comprensión intuitiva de sus capacidades y limitaciones.

El sistema fue construido íntegramente sobre ROS 2 Humble, aprovechando su modelo de comunicación *Pub/Sub* para implementar una arquitectura distribuida de seis nodos que separa claramente las responsabilidades de simulación, comunicación, arbitraje, autonomía y visualización. A lo largo de su desarrollo iterativo, se abordaron y resolvieron múltiples desafíos técnicos relacionados con la estabilidad del control, la fidelidad del canal acústico y la coordinación del brazo robótico.

2. Estado del Arte

2.1 Simulación de AUV en ROS 2

El ecosistema ROS 2 ofrece diversas opciones para la simulación de vehículos submarinos. El paquete UUV Simulator [2], originalmente desarrollado para ROS 1, proporciona modelos hidrodinámicos detallados en Gazebo, pero su complejidad de configuración y los requisitos computacionales de la simulación física tridimensional completa lo hacen poco adecuado para iteración rápida. Stonefish [3] ofrece un simulador más ligero con soporte nativo para ROS 2, pero sigue requiriendo modelos URDF, archivos de mundo y configuración de sensores complejos.

Estos simuladores priorizan la fidelidad física por encima de la accesibilidad y la rapidez de prototipado. En contraste, el enfoque adoptado en este trabajo privilegia la **fidelidad funcional**: se replican los efectos operativos (latencia, pérdida de paquetes, campo de visión limitado, inercia) sin necesidad de una simulación física tridimensional completa.

2.2 Comunicación Acústica Submarina

La propagación del sonido bajo el agua está gobernada por fenómenos de absorción dependiente de la frecuencia, dispersión geométrica y efectos de canal variable. La fórmula de Thorp [4] proporciona una aproximación práctica al coeficiente de absorción α en dB/km para una frecuencia f en kHz:

$$\alpha = \frac{0,11f^2}{1 + f^2} + \frac{44f^2}{4100 + f^2} + 2,75 \times 10^{-4}f^2 + 0,003 \quad (1)$$

La pérdida por transmisión (*Transmission Loss*, TL) combina la dispersión geométrica y la absorción:

$$TL = 20 \log_{10}(R) + \alpha \cdot R \cdot 10^{-3} \quad [\text{dB}] \quad (2)$$

donde R es la distancia en metros. La relación señal-ruido (SNR) resultante se calcula como $SNR = SL - TL - NL$, siendo SL el nivel de fuente y NL el nivel de ruido ambiental [5].

2.3 Arquitectura de Subsunción

La arquitectura de subsunción, propuesta por Brooks [6], organiza el control robótico en capas de comportamiento con prioridad estricta. Las capas inferiores (comportamientos reactivos de supervivencia) pueden *suprimir* las salidas de capas superiores (comportamientos orientados a objetivos). Este paradigma resulta especialmente apropiado para entornos dinámicos e impredecibles como el fondo marino, donde la evasión de obstáculos debe tener prioridad absoluta sobre cualquier objetivo de misión.

3. Hipótesis y Diseño

La hipótesis central de este trabajo es que un **simulador de AUV ligero, basado exclusivamente en comunicación Pub/Sub de ROS 2 y con renderizado 2.5D**, puede reproducir las restricciones operativas esenciales de un vehículo submarino real —latencia acústica, percepción limitada, inercia hidrodinámica— con la fidelidad funcional suficiente para validar algoritmos de autonomía y entrenar operadores, sin requerir la complejidad de un simulador físico tridimensional completo.

El diseño se fundamenta en tres premisas:

1. **Separación estricta simulación-control:** El motor de simulación publica *ground truth* en tópicos aislados (`/sim/*`), y un nodo puente introduce delay antes de republicarlos como percepción del AUV (`/auv/*`).

2. **Comunicación realista como requisito, no como opción:** Todos los comandos del operador deben atravesar el canal acústico simulado. No existen atajos de comunicación directa.

3. **Seguridad por subsunción:** Los comportamientos de evasión de obstáculos y *failsafe* de batería pueden anular tanto la autonomía como el control manual.

4. Metodología

4.1 Arquitectura General del Sistema

El sistema se compone de seis nodos ROS 2 propios, un nodo de visualización externo (RViz 2) y un nodo de teleoperación ejecutado en terminal independiente. La Figura 1 presenta la topología completa.

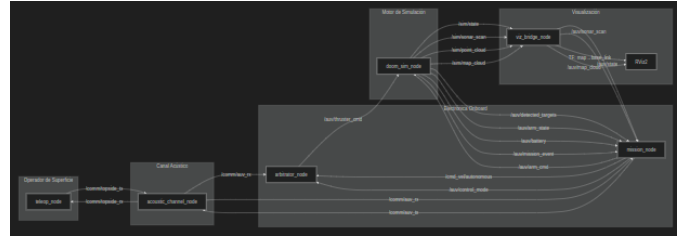


Figura 1: Topología ROS 2 del sistema (`rqt_graph`). Los nodos se agrupan en cinco dominios funcionales: Topside, Canal Acústico, Onboard, Simulador y Visualización.

La Tabla 1 resume los nodos y sus responsabilidades.

Cuadro 1: Nodos ROS 2 del sistema y sus roles.

Nodo	Responsabilidad
<code>doom_sim_node</code>	Motor de simulación: física, renderizado Pygame, publicación de sensores
<code>teleop_node</code>	Interfaz de operador en terminal con dashboard
<code>acoustic_channel_node</code>	Canal bidireccional con latencia y pérdida de paquetes
<code>arbitrator_node</code>	Multiplexor de comandos Manual/Auto
<code>mission_node</code>	Lógica de autonomía onboard con máquina de estados
<code>viz_bridge_node</code>	Puente con delay para RViz 2 (TF + PointCloud)

La decisión de utilizar exclusivamente tópicos Pub/Sub — sin Servicios ni Acciones ROS 2— responde a la naturaleza fundamentalmente asíncrona del dominio. Un Servicio ROS 2 es bloqueante: el cliente espera la respuesta del servidor. En un escenario donde la comunicación puede tardar segundos o perderse, este patrón provocaría bloqueos inaceptables en el bucle de control. Las Acciones, aunque asíncronas con soporte de *feedback*, introducirían complejidad de gestión de estados que se resuelve de forma más directa mediante la máquina de estados del nodo de misión combinada con eventos en tópicos.

4.2 Motor de Simulación

El nodo `doom_sim_node` ejecuta un bucle principal a 20 Hz que integra tres subsistemas: la física subacuática, el motor de renderizado y la interfaz ROS 2.

4.2.1 Motor de Física

La física modela un AUV con tres grados de libertad controlables: avance lineal, rotación en yaw y movimiento vertical. En lugar de asignar velocidad directamente desde los comandos recibidos, el sistema aplica un modelo de **inercia hidrodinámica acumulativa**. Los comandos del `/auv/thruster_cmd` (tipo Twist) se interpretan como fuerzas que aceleran el vehículo:

```
1 # Aceleracion: los comandos acumulan velocidad
2 self.linear_vel += cmd.linear.x * FORCE_LINEAR
3 self.angular_vel += cmd.angular.z * FORCE_ANGULAR
4 self.z_vel += cmd.linear.z * FORCE_Z
5
6 # Drag hidrodinámico: decaimiento por tick
7 self.linear_vel *= WATER_DRAG_LINEAR # 0.92
8 self.angular_vel *= WATER_DRAG_ANGULAR # 0.85
9 self.z_vel *= WATER_DRAG_LINEAR
```

Listing 1: Modelo de aceleración y drag hidráulico.

El factor de *drag* $d = 0,92$ implica que, en ausencia de fuerzas externas, la velocidad decae exponencialmente como $v(t) = v_0 \cdot d^t$, produciendo un movimiento suave con inercia perceptible. La colisión se resuelve mediante comprobación directa contra el mapa de celdas y verificación de proximidad con obstáculos, incluyendo una lógica que permite al vehículo escapar si ya se encuentra dentro del radio de colisión.

El modelo de batería implementa un consumo heterogéneo: un drenaje base de 0,001 unidades por tick más contribuciones proporcionales a la velocidad lineal ($|\dot{x}| \times 0,005$), angular ($|\dot{\theta}| \times 0,01$) y vertical ($|\dot{z}| \times 0,01$).

4.2.2 Motor de Renderizado

El renderizado emplea la técnica de *raycasting* [7], donde 120 rayos se lanzan desde la posición del AUV en un arco de 60° ($FOV = \pi/3$). Para cada rayo, se avanza iterativamente en incrementos de 5 píxeles hasta detectar una pared o alcanzar la profundidad máxima (100 unidades). La distancia perpendicular se calcula aplicando corrección de *fisheye*:

$$d_{\perp} = d_{\text{raw}} \cdot \cos(\theta_{\text{player}} - \theta_{\text{ray}}) \quad (3)$$

La altura de la columna renderizada es inversamente proporcional a la distancia: $h = 21000/(d_{\perp} + \epsilon)$. Los obstáculos dinámicos (barcos, criaturas) y estáticos (arrecifes, minas) se renderizan como *sprites* billboard con formas diferenciadas por tipo. Los *targets* de recolección se dibujan como círculos con un indicador de orientación (línea roja).

Un efecto colateral del raycasting es la revelación progresiva del mapa (*fog of war*): cada celda atravesada por un rayo se marca como explorada en una matriz booleana `fog_map[100×100]`, que controla la visibilidad tanto en el minimap de sonar como en los datos de exportación.

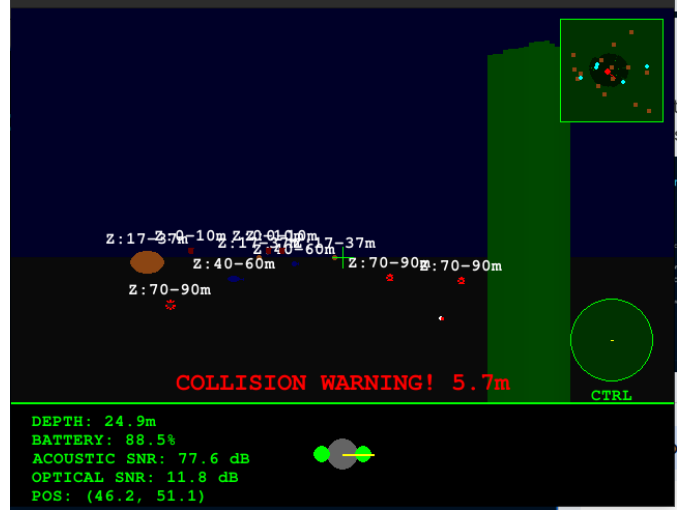


Figura 2: Interfaz del simulador DOOM AUV. La vista principal muestra el renderizado por raycasting 2.5D. El HUD inferior muestra profundidad, batería y SNR. El minimap superior derecho presenta la cobertura del sonar con *fog of war*. El brazo robótico con gripper aparece en la parte inferior central.

4.3 Canal de Comunicación Acústica

El nodo `acoustic_channel_node` implementa un canal bidireccional entre el operador de superficie (*topside*) y la electrónica onboard del AUV. Cada mensaje JSON que entra por `/comm/topside_tx` (downlink) o `/comm/auv_tx` (uplink) se encola con un retardo calculado como:

$$\tau = \frac{R}{c_s} + U(0,01, 0,05) \quad [\text{s}] \quad (4)$$

donde R es la distancia al AUV, $c_s = 1500$ m/s la velocidad del sonido en agua y $U(a, b)$ una componente uniforme aleatoria que modela el *jitter* del procesamiento. Adicionalmente, cada paquete tiene una probabilidad de pérdida del 5 %, independiente e idénticamente distribuida.

El módulo `CommunicationPhysics` implementa el modelo completo de SNR acústico según la Ecuación 2 con la absorción de Thorp (Ecuación 1) a $f = 25$ kHz y $SL = 185$ dB re $1 \mu\text{Pa}$. El SNR óptico se modela con una atenuación exponencial ($c = 0,08 \text{ m}^{-1}$, rango 100 m) correspondiente a agua oceánica clara. Un modelo simplificado de escintilación introduce cortes aleatorios del 5 % que simulan el efecto de ondas internas.

4.4 Arquitectura de Control

4.4.1 Arbitraje de Comandos

El nodo `arbitrator_node` opera como un multiplexor temporal: en cada tick de su bucle a 20 Hz, publica en `/auv/thruster_cmd` el último comando recibido de la fuente activa (manual o autónoma), siempre que dicho comando sea reciente ($< 0,5$ s). Si no hay comandos recientes, publica

un Twist nulo, lo que el drag hidráulico traduce en deceleración progresiva.

4.4.2 Subsunción de Seguridad

El nodo `mission_node` implementa una jerarquía de prioridades inspirada en la arquitectura de subsunción [6]. Las capas de seguridad operan **en todos los modos**, incluyendo el manual, y pueden forzar el cambio a modo autónomo:

1. **EMERGENCY_REVERSE** ($d < 1,5$ m): Retroceso puro durante 2 s. Prioridad máxima.
2. **AVOID_CRITICAL** ($d < 3,0$ m): Retroceso con rotación hacia el lado con mayor espacio libre, durante 3 s.
3. **BATTERY_FAILSAFE** (batería $< 10\%$): Fuerza ascenso de emergencia.
4. **Estados de misión** (EXPLORE, APPROACH, ALIGN, etc.): Solo activos en modo AUTO.

La dirección de evasión se determina en tiempo real comparando la suma de rangos del sonar en los hemisferios izquierdo y derecho:

```
1 mid = len(scan.ranges) // 2
2 left_sum = sum(scan.ranges[mid:])
3 right_sum = sum(scan.ranges[:mid])
4 turn_dir = 1.0 if left_sum > right_sum else -1.0
```

Listing 2: Determinación de la dirección de evasión.

4.5 Misión Autónoma

La autonomía onboard se estructura como una máquina de estados finita con seis estados principales (Figura 3).

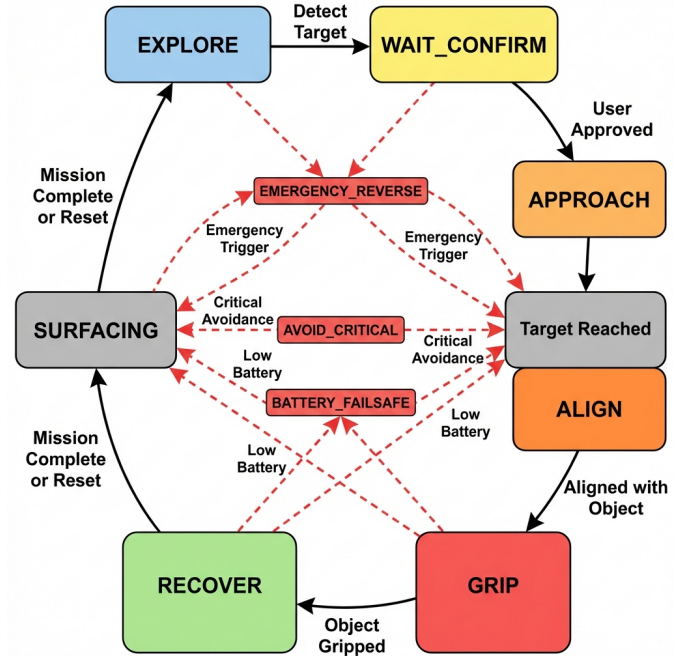


Figura 3: Máquina de estados de la misión autónoma. Las transiciones de seguridad (líneas rojas) pueden interrumpir cualquier estado.

EXPLORE: El AUV sigue un patrón *lawnmower* (barrido sistemático en zigzag) definido por una lista de waypoints en una rejilla de paso 20 unidades. La navegación emplea un controlador proporcional en *yaw* con ganancia $K_p = 0,5$, saturado en $\pm 1,0$ rad/s. La velocidad lineal se reduce a 0,2 cuando el error angular es grande ($|\psi_e| > 0,5$ rad) para evitar avance durante giros pronunciados.

WAIT_CONFIRM: Cuando el simulador publica un target visible en `/auv/detected_targets`, el `mission_node` transiciona a espera de confirmación, publicando telemetría actualizada. El operador debe enviar una confirmación (`confirm`) o rechazo (`reject`) a través del canal acústico. Los targets rechazados se almacenan para evitar redetección.

4.5.1 Control de Aproximación (APPROACH)

El estado APPROACH implementa un **controlador proporcional dual** que gobierna simultáneamente la velocidad lineal y la velocidad angular del AUV, con el objetivo de converger al target minimizando la distancia euclídea $d = \sqrt{\Delta x^2 + \Delta y^2}$ y el error de heading ψ_e .

El error de heading se calcula como la diferencia normalizada al intervalo $(-\pi, \pi]$:

$$\psi_e = (\text{atan2}(\Delta y, \Delta x) - \psi_{\text{auv}} + \pi) \bmod 2\pi - \pi \quad (5)$$

donde $\psi_{\text{auv}} = 2 \cdot \text{atan2}(q_z, q_w)$ se extrae del cuaternión de orientación. La ley de control angular aplica:

$$\omega = \text{clamp}(\psi_e \cdot K_\omega, -\omega_{\text{máx}}, \omega_{\text{máx}}) \quad (6)$$

con $K_\omega = 0,3$ y $\omega_{\text{máx}} = 0,5$ rad/s. Esta ganancia fue reducida respecto al valor original de 0,5 para prevenir giros agresivos que causaban sobrepasamiento y oscilaciones alrededor del target.

La velocidad lineal implementa un **esquema de conmutación con zona muerta angular**:

$$v = \begin{cases} 0 & \text{si } |\psi_e| > 0,2 \text{ rad} \\ \text{clamp}(0,3 \cdot d, 0,1, 0,5) & \text{en caso contrario} \end{cases} \quad (7)$$

La conmutación en $|\psi_e| = 0,2$ rad es crítica: **el AUV no avanza si no está apuntando al target**. Esto previene la deriva lateral que, sin inercia nula, acumularía velocidad perpendicular al heading deseado. La velocidad proporcional a la distancia ($0,3 \cdot d$) proporciona deceleración natural en la fase final, evitando el rebasamiento.

La transición a ALIGN se activa cuando se cumplen tres condiciones conjuntas: $d < 1,5$ m, $|\Delta z| < 2,0$ m (profundidad) y $|\psi_e| < 0,3$ rad. Un freno de emergencia ($v = 0$) se aplica incondicionalmente a $d < 0,5$ m para prevenir colisión con el target.

4.5.2 Alineamiento y Visual Servoing (ALIGN)

El estado ALIGN constituye el módulo de control más complejo del sistema. A diferencia de APPROACH, que opera en el espacio cartesiano del mundo, ALIGN utiliza **visual servoing basado en imagen** (*Image-Based Visual Servoing*, IBVS): el error se define directamente en coordenadas de pantalla (u, v) del target renderizado, y la acción de control se aplica sobre los actuadores del brazo robótico.

Congelación del cuerpo. Al entrar en ALIGN, el AUV detiene completamente su movimiento de cuerpo ($v = 0, \omega = 0$). Esta decisión de diseño evita un *loop* de realimentación indeseado: cualquier rotación del cuerpo desplaza la imagen del target en pantalla, lo que provocaría correcciones del brazo, que a su vez alterarían la posición relativa. Mantener el cuerpo estático desacopla los dos sistemas de control.

Control del brazo. El brazo se controla mediante comandos discretos (“left”, “right”, “up”, “down”, “rot_left”, “rot_right”) publicados a 10 Hz. Para determinar qué comando enviar, el algoritmo implementa una **política de prioridad con alternancia**:

```

1 # 1. Rotacion - Prioridad ALTA
2 if abs(diff_deg) >= 5.0:
3     if abs(diff_deg) >= 15.0 or tick_even:
4         cmd = "rot_right" if diff_deg > 0
5         else "rot_left"
6 # 2. Posicion en X (pantalla)
7 elif abs(cur_x - target_x) > 5: # px
8     cmd = "right" if cur_x < target_x
9     else "left"
10 # 3. Posicion en Y (pantalla)
11 elif abs(cur_y - target_y) > 5: # px
12     cmd = "down" if cur_y < target_y
13     else "up"
14 else:
15     pos_aligned = True

```

Listing 3: Bucle de visual servoing del estado ALIGN.

La rotación del gripper se computa como diferencia angular normalizada entre la orientación absoluta del target (extraída del cuaternión) y el ángulo actual del gripper:

$$\Delta\theta = (\theta_{\text{target}} - \theta_{\text{gripper}} + 180) \text{ mód } 360 - 180 \quad (8)$$

La política de alternancia (*tick_even*) permite que, cuando la rotación está “suficientemente cerca” ($|\Delta\theta| < 15$) pero no “perfecta” ($|\Delta\theta| \geq 5$), el sistema alterne entre correcciones de rotación y posición en *ticks* pares e impares. Esto evita que el brazo quede bloqueado en un ciclo de correcciones puramente rotacionales.

Avance lento (*creep forward*). Solo cuando la posición y rotación del brazo están alineadas pero la distancia 3D al target es $> 1,2$ m, el sistema aplica un avance lineal mínimo de $v = 0,1$, acercando el cuerpo al objeto sin perder la alineación. Esta lógica actúa como un controlador *bang-bang* de proximidad.

La Tabla 2 sintetiza los parámetros de control de ambos estados.

Cuadro 2: Parámetros de los controladores de APPROACH y ALIGN.

Estado	Parámetro	Valor	Unidad
APPROACH	K_ω (heading)	0.3	–
APPROACH	$\omega_{\text{máx}}$	0.5	rad/s
APPROACH	Rango v	[0.1, 0.5]	m/s
APPROACH	Zona muerta angular	0.2	rad
APPROACH	Dist. transición	1.5	m
ALIGN	Tol. rotación (estricta)	5.0	
ALIGN	Tol. rotación (alternancia)	15.0	
ALIGN	Tol. posición pantalla	5	px
ALIGN	v creep	0.1	m/s
ALIGN	Dist. grip (d_{3D})	1.2	m

4.5.3 Verificación y Cierre del Gripper (GRIP)

La transición de ALIGN a GRIP se activa cuando las tres condiciones son simultáneamente verdaderas: $|\Delta\theta| < 5$, posición centrada en pantalla ($|u_e| < 5$ px y $|v_e| < 5$ px), y $d_{3D} < 1,2$ m. Tras la transición, el módulo de renderizado verifica de forma independiente tres condiciones: distancia 3D $< 1,5$ m, error angular del gripper < 30 , y distancia en pantalla < 50 px. Si la captura falla, el sistema regresa a ALIGN paso 0 y reabre el gripper para reintentar.

4.6 Entorno Procedural

El MapManager genera un mundo submarino procedural sobre un mapa de 100×100 celdas. La Tabla 3 detalla la distribución de obstáculos.

Cuadro 3: Distribución de obstáculos y targets.

Tipo	N	Prof. (m)	Dinámica	ID Color
Barcos	10	0–10	Sí	Rojo (3)
Criaturas	10	40–60	Sí	Azul (4)
Arrecifes	15	17–37	No	Marrón (5)
Minas	15	70–90	No	Gris (6)
Targets	5	5–95	No	Cyan

5. Proceso de Desarrollo Iterativo

El sistema fue desarrollado mediante un proceso iterativo de refinamiento continuo que abarcó más de quince ciclos de desarrollo. A continuación se documentan los principales hitos y bloqueos técnicos resueltos, organizados cronológicamente.

5.1 Fase 1: Arquitectura Base

La primera iteración estableció los componentes fundamentales: el nodo de simulación con renderizado por *raycasting*, el *teleop_node* con comunicación directa mediante tópicos Twist, y un mapa estático con paredes fijas. En esta fase, el control manual publicaba directamente en el tópico */cmd_vel*, sin canal acústico intermedio. La física asignaba la velocidad de forma instantánea, sin inercia.

5.2 Fase 2: Comunicación Acústica y Dashboard

Se introdujo el *acoustic_channel_node* como intermediario obligatorio para toda la comunicación operador-AUV. El *teleop_node* fue rediseñado para empaquetar comandos como JSON y transmitirlos por */comm/topside_tx*. El dashboard de terminal se implementó con códigos ANSI para mostrar telemetría en tiempo real.

Un bloqueo importante fue la detección de la **desconexión del enlace acústico**: el dashboard mostraba “DISCONNECTED” intermitentemente porque la verificación de tiempo de último paquete no se actualizaba correctamente. La solución consistió en actualizar *last_packet_time* ante *cualquier* JSON válido recibido, independientemente de su contenido.

5.3 Fase 3: Autonomía y Evasión de Obstáculos

La implementación del *mission_node* con la máquina de estados y la arquitectura de subsunción requirió múltiples iteraciones. El primer problema crítico fue que el **auto-piloto no tomaba el control** cuando era necesario: las capas de seguridad no forzaban el modo AUTO en situaciones de emergencia. La solución fue añadir la publicación explícita en */auv/control_mode* desde las capas de seguridad.

5.4 Fase 4: Control del Brazo y Estabilización

La fase de alineamiento del brazo robótico para la captura de targets fue la más problemática y requirió al menos seis ciclos de refinamiento:

- **Gripper desalineado**: El gripper se posicionaba consistentemente a la izquierda del target. Se refinó la lógica de *visual servoing* para equilibrar rotación y ajuste posicional.
- **Distancia de captura**: Inicialmente fijada en 3 m, resultaba excesiva. Se redujo iterativamente a 1.5 m (verificación en *renderer.py*) y 1.2 m (transición en *mission_node*).

- **Oscilación vertical**: El controlador de profundidad con ganancia $K_p = 1,0$ causaba oscilaciones verticales que impedían la captura. Se redujo a $K_p = 0,3$ para un control más suave.
- **Velocidad del brazo**: La velocidad de rotación y desplazamiento del brazo se ajustó para permitir un control preciso sin sobrepasamiento.
- **Gripper que no cerraba**: Se relajó la tolerancia angular de 5 a 30 para permitir la captura con alineamientos aproximados, más realistas operativamente.

5.5 Fase 5: Realismo Acústico y Exportación 3D

En la fase final, se eliminaron los canales de comunicación directa residuales, asegurando que el 100 % del control manual pasara por el canal acústico con latencia y pérdida de paquetes. Se implementó la exportación del mapa explorado en formato OBJ (Wavefront) como mesh 3D, complementando los formatos PCD y PNG ya existentes.

6. Pruebas y Resultados

6.1 Protocolo de Pruebas

Las pruebas se ejecutaron en un entorno Ubuntu 22.04 LTS con ROS 2 Humble Hawksbill, Python 3.10 y Pygame 2.x. El sistema se inicia mediante el archivo launch:

```
1 $ colcon build
2 $ source install/setup.bash
3 $ ros2 launch doom_auv_sim doom_sim.launch.py
4 # En terminal separado:
5 $ ros2 run doom_auv_sim teleop_node
```

Listing 4: Secuencia de ejecución.

6.2 Resultados Funcionales

La Tabla 4 resume los resultados obtenidos en las pruebas del sistema completo.

Cuadro 4: Resultados de las pruebas funcionales.

Métrica	Valor	Estado
Targets recolectados (auto)	5/5	✓
Evasión de obstáculos	Funcional	✓
Battery failsafe	Activa < 10 %	✓
Comunicación acústica	Delay + 5 % loss	✓
Fog of war	Progresivo	✓
Exportación PCD/PNG/OBJ	Post-misión	✓
Frecuencia de simulación	20 Hz estable	✓
Frecuencia de control	10 Hz estable	✓

6.3 Análisis del Control de Profundidad

La reducción de la ganancia del controlador de profundidad de $K_p = 1,0$ a $K_p = 0,3$ eliminó las oscilaciones verticales observadas en versiones anteriores. El error de profundidad en estado estacionario se mantiene por debajo de $\pm 0,5$ m, lo cual

es suficiente para la transición de APPROACH a ALIGN (tolerancia de 2.0 m).

6.4 Análisis del Canal Acústico

El delay medio observado se sitúa entre 70 y 300 ms dependiendo de la distancia al punto de partida. La pérdida de paquetes del 5 % introduce saltos perceptibles en el dashboard de telemetría sin comprometer la estabilidad del sistema, gracias al diseño asíncrono de la arquitectura.

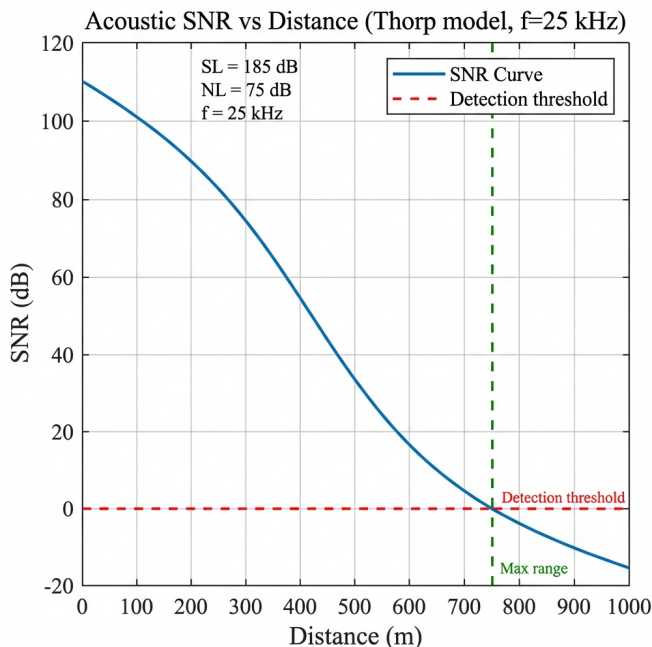


Figura 4: SNR acústico calculado en función de la distancia usando la fórmula de absorción de Thorp a 25 kHz.

7. Conclusiones y Trabajo Futuro

El presente trabajo ha demostrado la viabilidad de un simulador de AUV ligero construido sobre ROS 2 que reproduce las restricciones operativas esenciales del dominio submarino. El sistema integra con éxito un motor de renderizado por *raycasting*, un canal de comunicación acústica con modelo de propagación basado en Thorp, una arquitectura de control con jerarquía de subsunción para seguridad, y un módulo de autonomía onboard capaz de completar misiones de exploración y recolección.

El proceso de desarrollo iterativo puso de manifiesto la importancia del *tuning* de parámetros en robótica: la estabilidad del control de profundidad, la tolerancia de captura del gripper y la velocidad del brazo robótico requirieron múltiples ciclos de ajuste empírico. La documentación de estos ciclos constituye un aporte pedagógico valioso, ya que ilustra la realidad del desarrollo en robótica más allá de la arquitectura teórica.

7.1 Limitaciones Identificadas

- **Navegación reactiva:** La ausencia de un planificador global (e.g., Nav2 con A*) limita la capacidad del AUV para rodear obstáculos complejos. La navegación en línea recta con evasión temporal puede resultar en trayectorias subóptimas o bloqueos en configuraciones geométricas adversas.
- **Pérdida de paquetes i.i.d.:** El modelo de pérdidas independientes no captura la correlación temporal observada en canales acústicos reales. Un modelo de Markov (Gilbert-Elliott) proporcionaría mayor realismo.
- **Planificación 2D:** Tanto Nav2 como la mayoría de planificadores ROS 2 están diseñados para navegación 2D, lo cual limita su aplicabilidad directa a un AUV que opera en tres dimensiones.

7.2 Trabajo Futuro

- Integración de Nav2 para planificación de trayectorias 2D horizontales, manteniendo el controlador de profundidad actual.
- Modelo de canal acústico correlacionado (Gilbert-Elliott) y aplicación activa del shift Doppler a los datos de control.
- Extensión a escenarios multi-AUV cooperativos con protocolos TDMA de acceso al canal acústico.
- Adición de modelos de corrientes marinas que afecten a la navegación y a la propagación acústica.
- Integración con `slam_toolbox` para cartografía simultánea a partir del sonar simulado.

Referencias

- [1] M. Stojanovic y J. Preisig, "Underwater Acoustic Communication Channels: Propagation Models and Statistical Characterization," *IEEE Commun. Mag.*, vol. 47, no. 1, pp. 84–89, 2009.
- [2] M. M. M. Manhaes, S. A. Scherer, M. Voss, L. R. Douat y T. Rauschenbach, "UUV Simulator: A Gazebo-based package for underwater intervention and multi-robot simulation," *Proc. IEEE OCEANS*, 2016.
- [3] M. Cieřlak, "Stonefish: An Advanced Open-Source Simulation Tool Designed for Marine Robotics, with a ROS Interface," *Proc. IEEE OCEANS*, 2019.
- [4] W. H. Thorp, "Analytic Description of the Low-Frequency Attenuation Coefficient," *J. Acoust. Soc. Am.*, vol. 42, no. 1, p. 270, 1967.
- [5] R. J. Urick, *Principles of Underwater Sound*, 3ª ed. Peninsula Publishing, 1983.
- [6] R. A. Brooks, "A Robust Layered Control System for a Mobile Robot," *IEEE J. Robot. Autom.*, vol. 2, no. 1, pp. 14–23, 1986.

- [7] Lode Vandevenne, “Raycasting Tutorial,” <https://lodev.org/cgtutor/raycasting.html>, accessed Feb. 2026.
- [8] Open Robotics, “ROS 2 Documentation: Humble Hawksbill,” <https://docs.ros.org/en/humble/>, 2022.
- [9] M. Quigley *et al.*, “ROS: an open-source Robot Operating System,” *Proc. ICRA Workshop on Open Source Software*, 2009.
- [10] S. Macenski *et al.*, “The Marathon 2: A Navigation System,” *Proc. IEEE/RSJ IROS*, 2020.